

LLM Science Exam

Prompt Engineering Hackathon

Organised by Inceptez | June 2026

Competition:	Kaggle - LLM Science Exam
Leaderboard:	https://kaggle-llm-science-exam-hackathon-7qwbrayanhpdrixqw8bmra.streamlit.app/
Allowed Model:	Qwen 2.5 2.5B via Ollama (qwen2.5:2.5b) -- no other models permitted
Metric:	Mean Average Precision @ 3 (MAP@3)
Submission:	Upload submission.csv to the Streamlit leaderboard above

1. Problem Statement

You are given a set of challenging science questions, each with five answer choices labelled A through E. Only one answer is correct. Your task is to engineer a prompt that makes a local Large Language Model (LLM) select the correct answer as often as possible.

This hackathon is an exercise in prompt engineering - the art of writing instructions that get the best possible behaviour from a fixed model. You are NOT allowed to fine-tune, change model weights, or use a different model. The only thing that changes is the prompt.

2. Dataset

The dataset originates from the Kaggle competition 'LLM Science Exam'. Questions were generated by GPT-3.5 from Wikipedia passages and cover physics, chemistry, biology, earth science, and mathematics.

Download instructions:

- Go to: <https://www.kaggle.com/competitions/kaggle-llm-science-exam/data>
- Accept the competition rules if prompted.
- Download train.csv and test.csv.
- Place both files inside the data/ folder of the project.

After placing the files, run the split script once:

```
python prepare_splits.py
```

This automatically creates the following structure in data/:

```
data/
  train.csv          <- Downloaded from Kaggle (200 rows, with answers)
  test.csv           <- Downloaded from Kaggle (200 rows, no answers)
  train_examples.csv <- AUTO-SPLIT: 150 questions WITH answers [your dev set]
  test_without_answers.csv <- AUTO-SPLIT: 50 questions, no answers [val set]
  sample_submission.csv <- Shows the required output format
```

Use train_examples.csv (150 questions with visible answers) to develop and test your prompt locally. Your final submission is scored on the hidden 50-question val set -- you will not see those answers.

3. Technical Setup

Step 1 - Clone the repository

```
git clone https://github.com/Laxminarayan/kaggle-llm-science-exam-hackathon
cd kaggle-llm-science-exam-hackathon
```

Step 2 - Install Python dependencies

```
pip install pandas requests
```

Step 3 - Install Ollama and pull the required model

Download Ollama from <https://ollama.com>, then run:

```
ollama serve          # keep this running in a separate terminal
ollama pull qwen2.5:2.5b # one-time download (~1.9 GB)
ollama list            # verify: should show qwen2.5:2.5b
```

Step 4 - Prepare the data split

```
python prepare_splits.py
```

4. Rules

ALLOWED

ALLOWED

ALLOWED

NOT ALLOWED

NOT ALLOWED

NOT ALLOWED

NOT ALLOWED

Edit the prompt variable in solution.py

Embed few-shot examples inside the prompt string

Use DSPy to automate prompt optimisation (see README)

Change the model -- must stay qwen2.5:2.5b

Modify MODEL, query_model, or evaluation code

Use external APIs or any model other than Qwen 2.5B

Look up answers manually or hard-code them in any way

5. How to Participate

Step 1 - Open solution.py and find the prompt variable

This is the ONLY section you are allowed to change. It is clearly marked at the top of solution.py:

```
# =====  
# PROMPT <-- Edit this. This is the only thing you change.  
# =====  
prompt = """\n  
Question: {question}  
A) {A}  
B) {B}  
C) {C}  
D) {D}  
E) {E}  
Answer:\n  
"""
```

Keep the placeholders {question}, {A}, {B}, {C}, {D}, {E} exactly as-is. They are automatically replaced with real question data at runtime.

Step 2 - Evaluate your prompt locally

```
python local_eval.py
```

Runs your prompt against train_examples.csv (150 questions with known answers) and prints MAP@3. Use this to iterate quickly without submitting.

Step 3 - Generate your submission file

```
python solution.py
```

Runs inference on the 200-question test set and writes submission.csv with your ranked predictions.

Step 4 - Upload to the Inceptez leaderboard

```
https://kaggle-llm-science-exam-hackathon-7qwbrayanhpdrxiqw8bmra.streamlit.app/
```

Enter your team name and upload submission.csv. Your MAP@3 score appears instantly. Only your BEST score is kept -- you can resubmit as many times as you like.

6. Submission Format

Your submission.csv must have exactly two columns. solution.py generates this automatically -- you do not need to format it manually.

```
id,prediction  
0,C A B D E  
1,A B C D E  
2,D A B C E
```

...

- id: the question identifier (must match test.csv)
- prediction: five letters separated by spaces, ranked from most to least likely. Only the top 3 positions affect your score.

See data/sample_submission.csv for a full working example.

7. Scoring -- Mean Average Precision @ 3 (MAP@3)

Only your top 3 predictions matter. For each question, points are awarded based on where the correct answer appears in your ranked list:

- **1st choice correct:** 1.00 points
- **2nd choice correct:** 0.50 points
- **3rd choice correct:** 0.33 points
- **None of top 3 correct:** 0.00 points

Your final MAP@3 is the average of per-question scores across all questions. A baseline prompt scores approximately 0.16 MAP@3 with a 2.5B model. Good prompt engineering can push this significantly higher.

8. Prompt Engineering Tips

- Define a clear role: "You are a physics professor with 20 years of experience..."
- Add an elimination strategy: "First eliminate clearly wrong answers, then pick the best."
- Constrain the output tightly: "Reply with ONLY a single capital letter. Nothing else."
- Add 2-3 worked examples directly inside the prompt string (few-shot learning).
- Use chain-of-thought: "Think step by step, then state your final answer."
- If using reasoning, end with a signal: "Therefore my answer is: [letter]"
- Use DSPy to automatically search for better prompt instructions (see README.md).

9. Useful Links

- Leaderboard (submit here):** <https://kaggle-llm-science-exam-hackathon-7qwbrayanhpdrxiqw8bmra.streamlit.app/>
- GitHub Repository:** <https://github.com/Laxminarayan/kaggle-llm-science-exam-hackathon>
- Kaggle Dataset:** <https://www.kaggle.com/competitions/kaggle-llm-science-exam/data>
- Ollama Download:** <https://ollama.com>
- DSPy (advanced):** <https://github.com/stanfordnlp/dspy>